

Dynamic Graph Regularization for Multi-Stream Concept Drift Self-Adaptation

Ming Zhou^{ID}, Graduate Student Member, IEEE, Jie Lu^{ID}, Fellow, IEEE, Pengqian Lu, and Guangquan Zhang^{ID}

Abstract—Concept drift is an inevitable problem in non-stationary stream environments, due to evolving data distributions. In practical applications, multi-stream is more complex than single-stream, yet they have received little attention. Addressing concept drift while mining correlations between streams poses a significant challenge. Research focuses on capturing correlations between streams using graph neural networks (GNNs), providing valuable insights. However, these methods fix the correlation graph structure after training, failing to adapt to the new data distributions with dynamic correlations during testing. To bridge this gap, we propose a novel framework named Multi-stream Self-adaptation based on Graph Regularization (MSGR). A new GNN architecture is proposed to capture deep spatio-temporal correlations and learn a correlation graph structure without pre-defined graphs. Each graph node represents a stream and the graph is constructed through Gumbel sampling and an adaptive matrix from stream pairs. Thus we attain a base high-performance GNN for multi-stream multi-step prediction. To adapt to the new data distribution during testing, we design a self-adaptation mechanism by assigning dynamic learning weights for newly arriving samples. Larger learning weights are assigned to relevant samples when drift occurs. The self-adaptation is accomplished by the sub-graph updating and the proposed graph regularization. Error-based drift detection is integrated, and when drift occurs, the weight for sub-graph updating increases by adjusting the regularization coefficient. Thus, MSGR maintains high self-adaptation performance and accurate prediction results consistently regardless of the type and degree of concept drift. Comprehensive testing on real-world and synthetic datasets shows that MSGR achieves state-of-the-art performance.

Index Terms—Concept drift, data streams, drift adaptation, graph neural networks (GNNs).

I. INTRODUCTION

CONCEPT drift, due to its potential to render models ineffective or lead to a decline in accuracy, has been a persistent challenge throughout the development of traditional machine learning [1]. It refers to the phenomenon where the data distribution changes over time [2]. For example, it could involve irregular changes in weather due to climate warming or shifts in

public interest toward investment products caused by changes in the economic landscape. However, maintaining consistent data distribution between training and test data is crucial for traditional machine learning models to sustain performance [3]. Unfortunately, the problem of concept drift is ubiquitous, especially in non-stationary data stream environments [4]. The widespread use of sensors, internet propagation, and the progress of modern society have led to the generation, update, and rapid change of streaming data in various areas. The unpredictability of concept drift, with its unknown magnitude and duration, further exacerbates this challenge [5].

Self-adaptation offers a valuable approach to address the concept drift problem, enabling the model to learn and adapt to new data distributions [6]. This can be achieved through either an informed or blind strategy [7]. In the informed strategy, the model receives feedback from the drift detection module, and updates are made correspondingly when drift is unambiguously detected. Various statistical thresholds, such as error rates and multiple hypothesis tests, are integrated into the detection module to detect the concept drift [8]. On the other hand, the blind strategy entails a set of updating or learning rules [9], allowing the models to capture and learn changes in the data stream without relying on a drift detection mechanism [10].

A majority of the existing research on concept drift adaptation primarily focuses on single-stream tasks [11]. However, limiting the scope to a single-stream environment proves impractical for real-world applications. Frameworks designed for multi-stream environments can cover a broader range of applications and better align with the data mining needs of modern society. Moreover, applying the single-stream adaptation method to multiple streams separately leads to inefficiency, as it ignores the differences and correlations among streams [12]. This inefficiency also results in the wasteful consumption of significant computing resources and is not conducive to performing complex tasks related to correlation in multi-stream settings [13].

When we discuss data-driven tasks in multi-stream with considering the correlation between streams, each data stream is no longer isolated, which helps us further mine regionalized data information for different problems in a flexible task space [14]. In addition to the fixed correlation between streams, such as location information, the correlation between drifts should not be overlooked. Drift-causing factors have the potential to impact one or more data streams, and they can be categorized as concurrent drift or delayed drift based on the time difference of their occurrence [11]. An example is shown in Fig. 1. The sensors on Roads 1, 2, and 3 generate three data streams. These sensors'

Manuscript received 10 August 2023; revised 25 March 2024; accepted 6 May 2024. Date of publication 15 May 2024; date of current version 27 September 2024. This work was supported in part by Australian Research Council (ARC) under Grant FL190100149, and in part by the Discovery project under Grant DP200100700. Recommended for acceptance by G. Wang. (Corresponding author: Jie Lu.)

The authors are with the Decision Systems and e-Service Intelligence Laboratory, Australian Artificial Intelligence Institute (AAIL), Faculty of Engineering and Information Technology, University of Technology Sydney, Ultimo, NSW 2007, Australia (e-mail: Ming.Zhou@student.uts.edu.au; Jie.Lu@uts.edu.au; Pengqian.Lu@student.uts.edu.au; Guangquan.Zhang@uts.edu.au).

Digital Object Identifier 10.1109/TKDE.2024.3401156

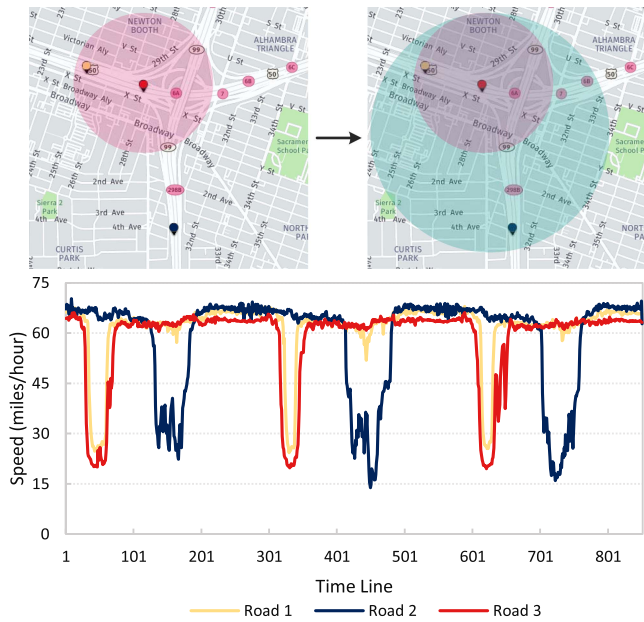


Fig. 1. An example for traffic multi-stream correlation. Three sensors on three roads generate three data streams. The location information indicates the base correlation between streams. When a traffic accident occurs near Road 1 and Road 3, all roads in the red circle are affected first (concurrent drift), and then the roads in the blue circle are affected soon after (delayed drift).

locations represent the base correlation of multi-stream. When a traffic accident occurs near Road 1 and Road 3, all roads in the red circle are affected first (concurrent drift), and then the roads in the blue circle are affected soon after (delayed drift). Notably, the practical scenarios are complex; for example, if Road 3 is closed temporarily, then the base correlation will fail. Therefore, correlation mining for multi-stream environments becomes a huge challenge for self-adaptation tasks in the concept drift field.

However, current methods designed for multi-stream concept drift adaptation have several limitations. For instance, some are not suitable for large-scale datasets, while some require specific source and target streams to function effectively [15], [16]. Moreover, certain approaches are limited to handling specific types of drift, which restricts their applicability [11]. Recently, some methods use graph neural networks (GNNs) to capture correlations between streams [17], [18], [19], but their static model fails to consider that the data distribution may change caused by concept drift in future data samples.

To fill these gaps, we propose a concept drift self-adaptation framework for multi-stream based on dynamic graph regularization, named Multi-stream Self-adaptation Graph Regularization (MSGR). MSGR can cover any drift type and is easy to apply to large-scale datasets. The adaptation process we design links the sub-graph updating with the degree of concept drift. Intuitively, the larger leaning weight should be assigned on the sub-graph updating when the drift is detected. In this paper, we exploit graph structure and a graph neural network to capture the correlation among streams. When drift happens, which potentially indicates the change of this correlation, the graph structure and

the network should be adjusted correspondingly. However, the correlation graph structure learned in the offline training process might still be informative, and it is also a resource wasting to totally discard it. Thus, we propose a graph regularization to sub-graph updating for the graph neural network, which helps to capture the new correlation change caused by drifts.

Two aspects need to be considered when designing our framework: the performance of correlation graph structure construction and the dynamic response capability to the unknown drift. Specifically, for the former, we exploit a sampled graph and an adaptive matrix in our designed novel graph neural network to help capture deep spatial-temporal correlation among streams. Each node in the graph refers to a stream. This network learned a correlation graph structure based on the historical data distribution without any pre-defined graphs in the offline training stage. When the offline training process is completed, we use this GNN as the base prediction model for the multi-stream multi-step prediction task in the online testing. To detect the concept drift in the online testing stage, we use the loss of the base model on the validation set as the drift detection threshold. For the latter, to dynamically adapt to the distribution change on testing time, sub-graph updating with graph regularization and real-time loss monitoring are performed all the time. The sub-graphs are updated by newly multi-stream samples and the graph regularization weight is assigned dynamically according to the monitoring result. To calculate this weight, a dynamic k NN graph and a dynamic sampled graph are generated by newly arriving data in the sliding window to achieve this function.

Our contributions are as follows:

- This paper proposes a novel concept drift self-adaptation framework based on dynamic graph regularization for multi-stream, named MSGR. The graph-based regularization assigns dynamic learning weight for sub-graph updating according to the drift detection result, which can help the model adapt to the concept drift in multi-stream more effectively and targeted.
- This paper learns a correlation graph structure by our designed novel graph convolutional layer to capture the deep dependencies between streams. This graph structure is realized by embedding a sampling graph from a multi-stream historical data distribution and calculating an adaptive transition matrix based on each node pair.
- This paper conducts extensive experiments on three large-scale real-world datasets and synthetic datasets, and the results show that MSGR can achieve significant improvement over state-of-art baseline methods.

The paper is organized as follows. The related work is presented in Section II. Section III covers the preliminaries and the problem definition. We present the proposed MSGR in detail in Section IV. The comprehensive experiments to evaluate our solution are given in Section V. We compare the prediction performance of MSGR and state-of-art baselines on real-world and synthetic datasets. The ablation studies of different parameters and components are also reported in this section. Section VI concludes our study and discusses suggestions for future research.

II. RELATED WORK

To succinctly outline the motivation behind our research, this section solely focuses on concept drift adaptation and graph neural networks for multi-stream prediction.

A. Concept Drift Adaptation

After obtaining a base model, how to adjust it to solve the concept drift problem during the testing stage is the core of developing an adaptation method. Two strategies usually motivate researchers to design the adaptation framework. When a specific concept drift detection module is embedded in the method, we call it informed adaptation; otherwise, we call it blind adaptation [1].

Many adaptation methods are developed based on these two strategies [13]. For informed adaptation methods, the drift can be detected by error, the differences between distribution, or multiple hypothesis tests [8]. The sliding window is a classic way to achieve the detection function. The ADWIN [20] uses an adaptive window to detect data distribution differences according to the defined statistic properties. The Drift Detection Method [21], Wilcoxon rank Sum Test Drift detector [22], and One Class Drift Detector [23] are also constructed on the window strategy. A targeted improvement on traditional machine learning algorithms is also a common method. Dynamic Clustering Forest [24] improves classification accuracy through multiple clustering trees on the basis of clustering and voting. The drift detection is achieved by finding clustering errors [25]. [26] uses the Kullback-Leibler (KL) divergence method to find concept drift.

If we apply the blind strategy to adapt concept drift, the detection module is no longer needed. The Very Fast Decision Tree classifier [27] and a series of similar algorithms ([28], [29]) use online learning to improve the traditional decision tree to adapt to the rapidly changing data stream. [30] also uses online learning to achieve self-adaptation, which utilizes an online ensemble classifier to accommodate concept drift, while self-bagging is used to alleviate the class imbalance problem. Furthermore, some methods which use deep learning frameworks achieve good performance, such as [31]. In recent years, fuzzy techniques applied to adaptation framework construction have been proven to be effective against certain types of drift [32]. The difficulty of high computational costs caused by the complex framework design of deep learning is also being gradually overcome [33].

These existing frameworks are mostly designed for single-stream scenarios. When the tasks are set in multi-stream scenarios, applying these methods to multiple data streams separately is a direct solution. But the calculation cost increases significantly with the increase in the streaming data scale. The high hardware cost and energy burden become factors that hinder this solution's wide application. For certain regional tasks, the correlation between streams plays a crucial role. Handling tasks separately for each stream compromises the overall logic of the algorithm framework and lead to inefficient execution. Scant research is dedicated to addressing this gap. The variance matrix is used to calculate the similarity of two streams in [34]. [11]

designs Bayesian neural network-based connectors to address the concurrent drift problem in a multi-stream. In [35], a self-adaptation framework based on GNNs is proposed to handle the multi-stream concept drift problem. These works motivate this research but further improvements are necessary to address the limitations and defects.

B. Graph Neural Networks for Multi-Stream Prediction

Graph Neural Networks (GNNs) have been proven to be an effective means to mine and construct the deep dependencies between multiple objects. A correlation graph structure between multiple nodes can be constructed in a non-euclidean space based on distance or other metrics, even for non-graph structured data. Currently, comprehensive GNN-based methods have been applied in many multiple series prediction areas, such as traffic prediction tasks, and have achieved good performance.

These methods calculate the correlation graph structure based on pre-defined graphs, which include the location information of sensors, especially if distance is used as the metric. DCRNN [36] embeds pre-defined graphs in a new diffusion convolutional structure. In Graph-WaveNet [17], pre-defined graphs are combined with an adaptive matrix to capture the deep dependency between streams. [37] uses a similar strategy to capture the spatial-temporal correlation among multiple series. STFGNN [38] focuses on the construction of the temporal graph and uses it to enhance the prediction performance. AST-GCN [39] introduces a latent network composed of a Laplace matrix to represent the spatial-temporal connections within the traffic data dynamically. Several methods can be used to improve the performance by modifying the network structure and adding some additional modules, such as GMAN [40]. Some researchers also try to improve the model performance through the pre-training mode. In [18], the authors build the pre-training model using the transformer technique to achieve better accuracy.

However, a significant challenge is that we cannot always obtain pre-defined graphs in advance, and if the task is not related to pre-defined graphs, the information contained is redundant. Several frameworks are proposed to handle this problem. In [41], two adaptive modules are designed to learn and represent the correlation information and mode parameters. GTS [42] constructs a sampled graph structure to simulate the historical data distribution and replace pre-defined graphs.

These GNN-based approaches served as inspiration for us to build spatial-temporal topologies from graph structures to explore the potential dependencies between data streams. However, none of these methods address the issue of concept drift in multi-stream settings. The graph structures in these models are solely constructed based on historical data distributions. In real-world applications, sudden or incremental drift changes are common occurrences, such as unexpected traffic accidents or extreme weather conditions. The offline prediction mode fails to consider the complexity and instability of streaming environments. Consequently, this serves as a key motivation for us to introduce the multi-stream self-adaptation method to tackle the concept drift challenge.

TABLE I
 SUMMARY OF NOTATIONS

Notations	Description
$\mathcal{S} = \{\mathcal{D}^1, \dots, \mathcal{D}^i\}$	A multi-stream contains i data streams.
$\mathcal{D} = \{T^1, \dots, T^j\}$	A data stream contains j time series.
$\mathcal{T}^j = \{X_t^j\}$	A time series in the data stream.
X^j	The feature value of the time series.
$t \in \mathbb{Z}^+$	The time step.
$\mathcal{P}$	The probability distribution.
Δ	The past window size.
η	The prediction time steps.
$\mathcal{G}$	The correlation graph structure of multi-stream.
$\mathcal{F}$	The predictor for multi-stream.
$\mathcal{S}^{tr}, \mathcal{S}^{val}, \mathcal{S}^{te}$	The training, validation and testing sets.

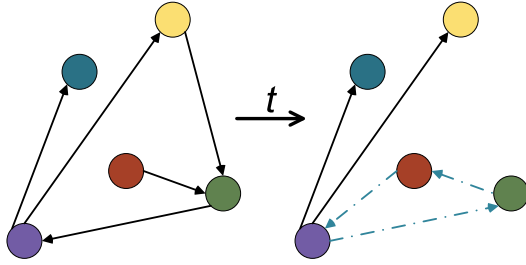


Fig. 2. The multi-stream correlation changes over time.

III. PRELIMINARIES

In this section, the definitions and our problem statement are introduced. We present a summary of some important notations in Table I for brevity.

Definition 1 (Multi-Stream). A multi-stream is denoted as $\mathcal{S} = \{\mathcal{D}^1, \mathcal{D}^2, \dots, \mathcal{D}^{N_i}\}$, where $\mathcal{D}^i = \{\mathcal{T}^1, \mathcal{T}^2, \dots, \mathcal{T}^{N_j}\}$ is a single data stream in $\mathcal{S}$. Each $\mathcal{D}$ consists of j feature time series, X_t^j denoted as the value of time series $\mathcal{T}^j$ at time step t .

The multi-stream environment is a significant challenge in the field of data mining. Our method is designed to handle the multi-series concept drift problem in multi-stream. Specifically, in our study, we consider that each data stream $\mathcal{D}^i$ in the multi-stream $\mathcal{S}$ contains the same feature time series. For example, multiple traffic monitor stations are placed in different areas of the city, and they monitor some same traffic features, such as speed, the number of vehicles, etc. using the sensors in these stations. Our research is based on the common features in these streams, where the same target feature series from multiple streams constitutes a multi-stream in our task.

Definition 2 (The Correlation in Multi-Stream). The correlation in multi-stream is denoted as $\mathcal{G}$. It comprises two parts, the correlation between streams and the correlation between concept drifts in different streams.

As shown in Fig. 2, in a multi-stream, the correlation structure $\mathcal{G}$ is in a dynamic state in streaming environments. The basic correlation between streams is usually formed by the location

distance between sensors, so the global correlation structure will be stable in most cases, but sub-graphs will be dynamic. Concept drift is the main reason causing this local change.

For example, a traffic accident at a certain intersection may cause congestion in adjacent road sections, and thunderstorm occurrence in a local area will also cause a significant increase in the overall humidity in the area. On the other hand, if there is a parade or a road closure partly for repairs, the correlation will also change locally.

Definition 3 (Concept Drift in Multi-Stream). Assume that $\mathcal{P}$ presents the data distribution. When $\mathcal{P}_{t+1}(\mathcal{S}) \neq \mathcal{P}_t(\mathcal{S})$, concept drift occurs at time step $t + 1$. If concept drift occurs in any stream of $\mathcal{S}$, which is denoted as the concept drift in multi-stream.

Definition 4 (Concept Drift Correlation in Multi-stream). Two cases of drift correlation exist in multi-stream based on the time difference. When concept drift occurs on more than one stream simultaneously, it is denoted as concurrent drift. If there is a time difference between these drifts, it is denoted as delayed drift. The detailed definition of the correlation between drifts in multi-stream is reported in [35].

Problem 1 (Learning for Multi-Stream at t -step with Dynamic $\mathcal{G}_t$). Given previous δ step information, and the correlation attributes $\mathcal{G}$, predict the next η -step for multi-stream, which can be formalized as

$$\mathcal{F}_t, \mathcal{G}_t = \arg \min_{f \in \mathcal{F}, \mathcal{G}} \ell(f(\mathcal{S}_{t-\Delta+1:t}, \mathcal{G}_{t-1}), \mathcal{S}_{t+1:t+\eta}). \quad (1)$$

where $\mathcal{G}_t$ is achieved by the sub-graph updating mode from $\mathcal{G}_{t-1}$ with every newly arrived sample in the online testing stage.

IV. METHODOLOGY

This section describes the detailed architecture of our proposed MSGR. The framework of MSGR is shown in Fig. 3. Our method is divided into two stages: the offline training stage and the online testing stage. To obtain a more accurate correlation graph structure, a novel graph neural network is designed to learn the historical data distribution and deep correlation between streams in the training stage. In the online testing stage, to overcome the concept drift problem, we propose a graph-based dynamic regularization, which cooperates with sub-graph updates to realize self-adaptation. The real-time prediction error rate is monitored to detect the occurrence of drift, and it is connected to the regularization process to capture local correlation changes more accurately in the multi-stream.

A. Offline Initialization

Before online testing, we need to build a base prediction model, so we design a graph neural network to capture deep spatio-temporal correlations between streams. After training, we can obtain an initialised multi-stream correlation graph structure and a GNN model as the base prediction model for the online testing stage.

1) Graph Structure Convolutional Layer: The use of pre-defined graphs is a common approach to represent the correlation among multi-stream data. These pre-defined graphs typically

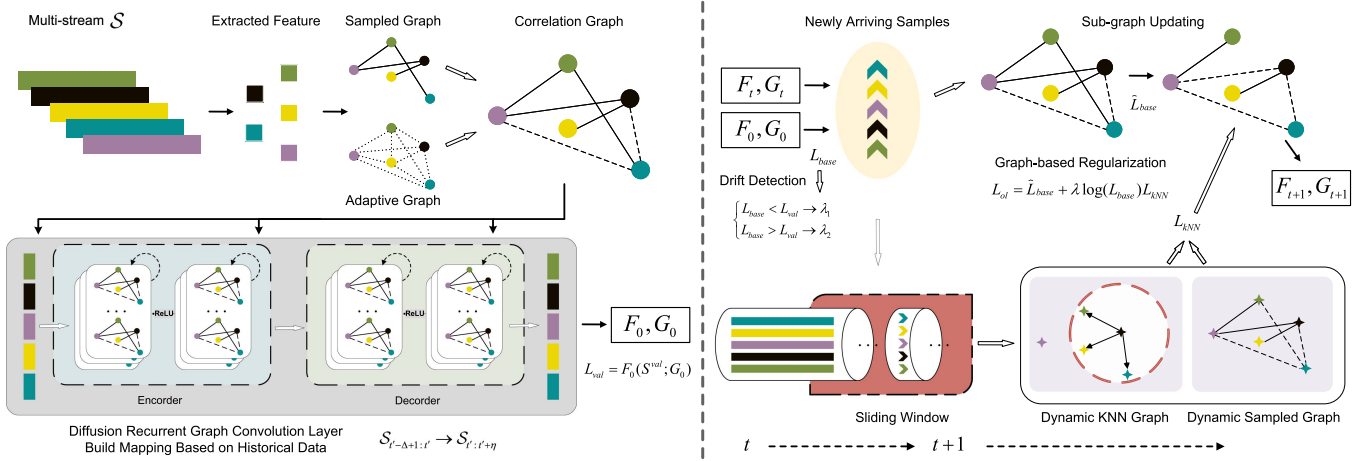


Fig. 3. Overview of the MSGR framework. The training stage, depicted on the left, involves embedding the sampled graph and adaptive graph into the Diffusion Recurrent Graph Convolutional layer using a sequence-to-sequence structure. This stage aims to obtain a base prediction model. The image on the right shows our self-adaptation method during the online testing stage. The sub-graphs are updated using newly arriving data over time, and at the same time, dynamic graph-based regularization helps this updating process to adapt to the new data distribution. Two dynamic graphs generated by the new data stored in the sliding window achieve regularization weight calculation. The weight coefficient is determined based on the specific drift situation.

include location information for sensors. However, pre-defined graphs may not always be available in practical scenarios for multi-stream environments. Moreover, if we solely rely on fixed information from a pre-defined graph without ensuring a strong relationship with the prediction task, the prediction results may show significant deviations [41].

To learn more rich and deep correlation information from historical data of the multi-stream, we proposed a novel graph structure convolutional layer. First, we simulate the historical data distribution of all streams based on sampling. However, a challenge exists here: the gradient cannot pass through the discrete graph structure after Bernoulli sampling. Motivated by [42], we employ the Gumbel-softmax trick [43], [44] to address this problem using a reparameterization process.

Specifically, a feature extraction network is applied to extract historical data feature information and compress the data size: $z^u = \text{FC}(\text{vec}(\text{Conv}(\mathcal{S}_{tr}^u)))$, where $\mathcal{S}_{tr}$ represents the entire training data. The historical data is convolved and vectorized along the time dimension, fully connected layers are used to reduce the dimension, and each stream is mapped to a vector. The training process is performed by batch data, but $\mathcal{S}_{tr}$ for sampling will be static to ensure the correlation structure learning is accurate.

After feature preprocessing, all data streams are combined in pairs of vectors (z^u, z^v) . To obtain the pairwise combination of vectors, concatenation is performed by two fully connected layers: $\mu_{(u,v)} \in [0, 1]$, which gives the correlation weight between these two streams based on their distribution.

The reparameterization trick is employed here to calculate the sampled graph structure between every pair of streams u and v , denoted as G_{Gum} . Perturbed Gumbel noises based on uniform distribution are generated which correspond to N dimensional vectors Ω . $\{\text{Gum}_i\}_{i \leq N}$, the i.i.d sequence of standard Gumbel random variables can be calculated using the following equation:

$$\text{Gum}_i = -\log(-\log(U_i)), U_i \sim U(0, 1). \quad (2)$$

Then we can get G_{Gum} :

$$G_{\text{Gum}} = \text{softmax} \left(\frac{\log\left(\frac{\mu_{(u,v)}}{1-\mu_{(u,v)}}\right) + (\text{Gum}_u - \text{Gum}_v)}{\delta} \right) \quad (3)$$

where δ represents the temperature parameter.

To ensure the performance of the sampled graph is robust, we utilize a self-adaptive transition matrix [17] and embed it into the convolutional layer, which can dynamically learn node pair correlation probabilities constructed by matrix calculations to capture the deep correlation between streams in the convolutional process.

The self-adaptive transition matrix is optimized by two randomly initialized node embedding dictionaries with learnable parameters $E_1, E_2 \in \mathbb{R}^{N \times d}$:

$$A_{\text{adp}} = \text{Softmax}(\text{ReLU}(E_1 E_2^T)) \quad (4)$$

E_1 and E_2 are the source node embedding and target node embedding respectively, where each row of E_1 represents a node embedding and d refers to the embedding dimension. The spatially dependent weights between source nodes and target nodes are derived by matrix multiplication. The ReLU activation function is used to eliminate weak connections. The softmax function can normalize the adaptive adjacency matrix. Therefore, the normalized adaptive adjacency matrix can be considered as a transition matrix that hides the diffusion process. By embedding the sampled graph and self-learned adaptive graph, we obtain the following graph convolutional learning layer based on the spatial correlation:

$$\mathcal{G} = \sum_{p=0}^P G_{\text{Gum}}^p \text{SW}_{p1} + A_{\text{adp}}^p \text{SW}_{p2} \quad (5)$$

where P is the finite steps of the diffusion process of graph signals and W_p is the graph convolutional parameters of p -th order.

2) *Temporal Convolutional Layer*: In MSGR, we utilize a sequence-to-sequence structure to undertake the multi-step prediction task: $S_{t-\Delta+1:t}$ to $S_{t+1:t+\eta}$. Our designed graph structure convolutional layer is embedded in a recurrent convolutional framework to complete spatio-temporal modelling and mapping tasks. At each time step t^* , the hidden state of the sequence-to-sequence model undergoes an update from $\mathbf{h}_{t^*-1}$ to $\mathbf{h}_{t^*}$. The input for this update consists of the values of S_{t^*} within a defined historical window size Δ . Specifically, the encoder part of the model recurrently updates from $t' - \Delta + 1$ to t' , with $\mathbf{h}_{t^*}$ serving as the representation of S_{t^*} . Subsequently, $\mathbf{h}_{t^*}$ is used to initialize the decoder. The hidden state then undergoes recurrent updates for each subsequent η -time step in the future, moving from $t' + 1$ to $t' + \eta$.

Many existing architectures can be applied here to serve this purpose. We use the Diffusion Convolutional Gated Recurrent Unit designed by [36] to complete the encoder-decoder function because it is more suitable for the directed graph structure. The diffusion process is achieved by a random walk on graph structure $\mathcal{G}$:

$$\Theta_\lambda \star_{\mathcal{G}} Z = \sum_{p=0}^P (\theta_{p,1}^\lambda (M_O^{-1} \mathcal{G})^p + \theta_{p,2}^\lambda (M_I^{-1} \mathcal{G}^T)^p) Z$$

where Θ is the filter to perform the diffusion convolutional operation on $\mathcal{G}$, M_O and M_I are the out-degree and in-degree matrix. Θ_λ for $\lambda = r, q, c$ are the model parameters for the corresponding filters. Then the graph diffusion module will replace the matrix multiplications in GRU [45]:

$$\begin{cases} r_{t^*} = \text{sigmoid}(\Theta_r \star_{\mathcal{G}} [S_{t^*} \parallel \mathbf{h}_{t^*-1}] + b_r), \\ q_{t^*} = \tanh(\Theta_q \star_{\mathcal{G}} [S_{t^*} \parallel (r_{t^*} \odot \mathbf{h}_{t^*-1}) + b_q], \\ c_{t^*} = \text{sigmoid}(\Theta_c \star_{\mathcal{G}} [S_{t^*} \parallel \mathbf{h}_{t^*-1}] + b_c), \\ \mathbf{h}_{t^*} = c_{t^*} \odot \mathbf{h}_{t^*-1} + (1 - c_{t^*}) \odot q_{t^*} \end{cases}$$

where r_{t^*} and q_{t^*} are the reset gate and update gate at time step t^* . Mean absolute error (MAE) between the prediction values $\mathcal{Y} \in \mathbb{R}^{\eta \times I \times J}$ and ground truth $\hat{\mathcal{Y}} \in \mathbb{R}^{\eta \times I \times J}$ in future η time steps is the loss function of the training process, which can be defined as:

$$\mathcal{L}_{base} = \mathcal{L}(\hat{\mathcal{Y}}, \mathcal{Y}) = \frac{1}{\eta I J} \sum_{t=1}^{\eta} \sum_{i=1}^I \sum_{j=1}^J |\hat{\mathcal{Y}}_{ijt} - \mathcal{Y}_{ijt}|, \quad (6)$$

where I is the number of streams, J is the dimensionality of the output.

In the training stage, we construct a novel graph structure convolutional layer and utilize a new GNN-based network to map the spatio-temporal correlation among multi-stream data. Upon completion of the training, we can obtain a GNN model $\{\mathcal{F}_0, \mathcal{G}_0\}$, which is used as the base prediction model for the multi-stream multi-step prediction task in the online testing stage. $\mathcal{G}_0$ includes the original correlation graph structure of multi-stream based on historical observations and contains two parts: global correlation and local correlation. Usually, the global correlation remains stable because the locations of the sensors are fixed. Local correlation refers to the associations between

different data streams that arise due to various factors. The local correlation may change over time or due to the occurrence of concept drift. For example, in relation to roads in different areas of a city, roads that are in close proximity have similar busy degrees. These roads will suffer from similar traffic jam risks if a traffic accident occurs on one of the roads. But the local correlation changes caused by concept drift are not contained in the base prediction model, so an online-based prediction mode is necessary. Also, the loss of $\{\mathcal{F}_0, \mathcal{G}_0\}$ on the validation set $\mathcal{L}_{val}$ will be the threshold for concept drift detection in the testing stage.

B. Online Dynamic Graph-Based Self-Adaptation

The static prediction model carries a high risk when faced with unknown changes in streaming data. The concept drift means that new data distributions occur in newly arriving samples. To tackle this issue, we propose a novel self-adaptation strategy, achieved through dynamic graph-based regularization, to adjust the model over time continuously. This approach enables the timely capture and learning of any new unknown changes caused by concept drift, ensuring accurate multi-step prediction results are provided simultaneously.

We aim to stabilise the global correlation and adjust the model with accurate sub-graph updating. The work in [35] was the first to handle the concept drift problem in multi-stream in this way using simple online learning in cooperation with a GNN pool and a dynamic validation set.

But the shortcomings of this method are also evident. On the one hand, the multi-model mechanism can partially avoid incorrect updates, but it consumes a considerable amount of storage resources, and the update process still lacks effective constraints. This means that it cannot adjust the degree of model updating based on the specific drift situation. On the other hand, many parameter settings dependent on the nature of the data also limit its applicability.

In our framework, these shortcomings have been addressed. First, our proposed new graph neural work can learn deep correlations between data streams during training, and the high-performance initial prediction model eliminates the need to store multiple GNN models. Second, the proposed graph-based dynamic regularization is applied to the sub-graph updating process based on different drift situations, effectively adapting the new data distribution. Meanwhile, real-time error rates are linked to the regularization computation to capture local correlation changes accurately. Moreover, the simplified parameter settings significantly enhance the model's applicability.

Our designed regularization will take full advantage of the original graph structure. We use the newly arriving multi-stream data stored in a sliding window $\mathcal{V}^\tau$ with size τ to generate a dynamic weighted directed k NN graph and a dynamic sampled graph separately. The computational process to generate a sampled graph is the same as it is in the training stage. The dynamic k NN graph can be defined as:

$$G_{kNN}^\tau = \text{top}_k f_{cos}(S^\tau), \quad (7)$$

Algorithm 1. MSGR Dynamic Self-Adaptation Process.

1: Input Multi-stream $\mathcal{S}$, the input sequence length for all streams is Δ , and the output length is η . The batch size of the multi-stream in the testing stage is 1 (The new samples will arrive one by one). The online sub-graph updating rate is e_{ol} , the epoch number is Q_m . The loss function is calculated by MAE, and the optimization algorithm is Adam [46]. The k for dynamic k NN graph. The regularization weight coefficients are λ_1, λ_2 for specific drift situations.

2: Initial The base prediction GNN model and correlation graph structure obtained from the training stage $\{\mathcal{F}_0, \mathcal{G}_0; \Theta_0\}$. The loss of $\mathcal{F}_0$ on the validation set is $\mathcal{L}_{val}$. The dynamic feature sliding window $\mathcal{V}^\tau$ is initialized by the historical data of $\mathcal{S}$, and the window size is τ .

for New samples of $\mathcal{S}$ arrive in each time step, $t = 1, 2, 3, \dots$ **do**

3: Online Prediction: $\mathcal{F}_t(S_{t-\Delta+1:t}^{te}; \mathcal{G}_{t-1}, \Theta_{t-1}) = S_{t+1:t+\eta}^{pre}$;
 // Give online prediction results for all streams

if $t > 1$ **then**

4: Obtain Ground Truth: $S_{t:t+\eta-1}^{true}$;

5: Update Dynamic Feature Window New arriving samples are spliced with the data in $\mathcal{V}^\tau$, and the old data at the remote end of the window is discarded;

6: Calculate Initial Model Loss
 $\mathcal{L}_{base} = \text{loss}(\mathcal{F}_0(S_{t-\Delta:t-1}^{te}; \mathcal{G}_0, \Theta_0), S_{t:t+\eta-1}^{true})$;

7: Calculate Current Model Loss:
 $\hat{\mathcal{L}}_{base} = \text{loss}(\hat{\mathcal{F}}_{t-1}(S_{t-\Delta:t-1}^{te}; \mathcal{G}_{t-1}, \Theta_{t-1}), S_{t:t+\eta-1}^{true})$;

8: Generate Dynamic k NN Graph and Sampled Graph:
 G_{kNN}^τ (see Eq.(7)), G_{Gum}^τ (see Eq.(3));

9: Calculate Dynamic Regularization Loss:
 $\mathcal{L}_{kNN}$ (see Eq.(8));

for $Q = 1, 2, \dots, Q_m$ **do** // Sub-graphs Updating under Dynamic Graph-based Regularization

if $\hat{\mathcal{L}}_{base} < \mathcal{L}_{val}$ **then**

10: Update under No Drift
 $\mathcal{G}_{t-1}, \Theta_{t-1} = \text{Adam}(\mathcal{L}_{ol}; \lambda_1(\text{Eq.}(11)), F_t)$;
 $\mathcal{G}_t = \mathcal{G}_{t-1}, \mathcal{F}_t = \mathcal{F}_{t-1}$;

else

11: Update under Concept Drift
 $\mathcal{G}_{t-1}, \Theta_{t-1} = \text{Adam}(\mathcal{L}_{ol}; \lambda_2(\text{Eq.}(11)), F_t)$;
 $\mathcal{G}_t = \mathcal{G}_{t-1}, \mathcal{F}_t = \mathcal{F}_{t-1}$;

end

end

end

where S_τ is the data in the sliding window, and k is the parameter of the k NN graph. G_{kNN} is updated over time with streaming data updating. The cross-entropy between the sampled graph and the dynamic k NN graph achieves the regularization function:

$$\mathcal{L}_{kNN} = \sum_{uv} -G_{kNN}^\tau \log G_{Gum}^\tau - (1 - G_{kNN}^\tau) \log(1 - G_{Gum}^\tau) \quad (8)$$

Then the new loss function for the online sub-graph updating process can be defined as:

$$\mathcal{L}_{ol} = \hat{\mathcal{L}}_{base} + \mathcal{L}_{kNN}, \quad (9)$$

where $\hat{\mathcal{L}}_{base}$ is the loss calculated by the current model $\{\mathcal{F}_{t-1}, \mathcal{G}_{t-1}\}$ using samples in the window.

But applying the regularization at the same weight on the original loss function for the whole learning process is unreasonable, as inappropriate regularization will have a negative impact on the

model. We design a dynamic regularization magnitude adjustment mechanism to address this problem, where λ is the weight coefficient for regularization depending on the drift detection result. The loss of the base model on the validation set $\mathcal{L}_{val}$ is used as the threshold for drift detection. The real-time error rate $\mathcal{L}_{base}$ calculated by the base model $\{\mathcal{F}_0, \mathcal{G}_0\}$ on the data in the window is monitored over time. When $\mathcal{L}_{base}$ exceeds $\mathcal{L}_{val}$, that indicates that the concept drift has occurred. According to the detection result, different weight coefficients λ_1, λ_2 are applied for no drift and drift detected situations, respectively. The online loss function can be defined as follows:

$$\mathcal{L}_{ol} = \hat{\mathcal{L}}_{base} + \lambda \mathcal{L}_{kNN}, \lambda \in \{\lambda_1, \lambda_2\} \quad (10)$$

To capture the local correlation change more accurately, we also link the real-time prediction error $\mathcal{L}_{base}$ to the regularization calculation, then the complete online loss function is as follows:

$$\mathcal{L}_{ol} = \hat{\mathcal{L}}_{base} + \lambda \log(\mathcal{L}_{base}) \mathcal{L}_{kNN}, \lambda \in \{\lambda_1, \lambda_2\} \quad (11)$$

Overall, dynamic self-adaptation is accomplished by the sub-graph updating process with the dynamic graph-based regularization: $\{\mathcal{F}_{t-1}, \mathcal{G}_{t-1}\} \rightarrow \{\mathcal{F}_t, \mathcal{G}_t\}$. Our design realizes the deep connection between the concept drift self-adaptation mechanism and the specific drift degree, which makes the online learning process more targeted and largely retains the model performance trained via historical information.

C. The MSGR Algorithm (Self-Adaptation Stage)

The pseudo-code of our designed MSGR dynamic self-adaptation process is shown in Algorithm 1. In Step 1, the parameter setting for the online testing stage is conducted. In Step 2, the testing initialization is undertaken, where the base prediction model containing the correlation graph structure $\{\mathcal{F}_0, \mathcal{G}_0\}$ is used to perform the prediction task, and the loss of $\{\mathcal{F}_0, \mathcal{G}_0\}$ on the validation set $\mathcal{L}_{val}$ is the threshold of the drift detection. The online testing stage starts in Step 3, where new samples arrive at each time step and the model provides the next η step prediction values of all streams. Our self-adaptation strategy, which involves sub-graph updating and graph-based regularization, is implemented to address the concept drift problem. Specifically, the ground truth is obtained in Step 4. Then, the newly available samples are stored and spliced in the sliding feature window in Step 5.

Steps 6 to 8 use the samples stored in the window to calculate different loss values and generate different dynamic graphs for regularization. In Step 6, the real-time error $\mathcal{L}_{base}$ is calculated by the original base model $\{\mathcal{F}_0, \mathcal{G}_0\}$. The model $\{\mathcal{F}_{t-1}, \mathcal{G}_{t-1}\}$ updated from the previous time step calculates the loss value $\hat{\mathcal{L}}_{base}$ in Step 7. Then in Step 8, a dynamic k NN graph and a sampled graph are generated separately. The cross-entropy $\mathcal{L}_{kNN}$ between these two graphs is calculated in Step 9. We apply the different weight coefficients λ_1 and λ_2 on the regularization to no drift and drift detected situations and the real-time error $\mathcal{L}_{base}$ is also linked to the regularization process. When $\mathcal{L}_{base} < \mathcal{L}_{val}$, this indicates that there is no concept drift has occurred. Then the sub-graph updating will be performed

TABLE II
SUMMARY OF DATASETS

Dataset	METR-LA	PEMS-BAY	WEATHER
Nodes	207	325	10
Samples	34272	52116	29639
Update Rate	5min	5min	1h

using $\mathcal{L}_{ol}$ under λ_1 in Step 10. When $\mathcal{L}_{base} < \mathcal{L}_{val}$, this indicates that the concept drift has occurred, and the updating process in Step 11 will be performed under λ_2 . Then the self-adaptation process of this time step $\{\mathcal{F}_{t-1}, \mathcal{G}_{t-1}\} \rightarrow \{\mathcal{F}_t, \mathcal{G}_t\}$ is completed.

V. EXPERIMENTS

We construct extensive experiments to verify the effectiveness of the proposed MSGR. The real-world datasets (traffic and weather areas) are used to demonstrate the prediction performance of our MSGR by comparing it with existing frameworks. The synthetic datasets are used to further verify the self-adaptation performance by testing the occurrence of sudden drift in multi-stream.

We first give detailed information of the datasets and baseline models used for the experiments, and then the related settings, experimental results, and ablation study are discussed in corresponding subsections.

A. Datasets and Baselines

Datasets. Following previous works [17], [35], [42], our main experiments are constructed on three large-scale real-world datasets from the traffic and weather areas. A summary of these datasets is shown in Table II.

- **METR-LA** is a dataset that captures the speed of traffic on the road network in LA County. The data is collected from loop detectors placed at 207 locations [47]. The dataset covers a duration of 4 months, from March to June in 2012 [36]. Traffic information is recorded at 5-minute intervals, resulting in a total of 34,272 time slices.
- **PEMS-BAY** is a publicly available dataset that captures traffic speed information from the Performance Measurement System (PeMS) of the California Transportation Agencies (CalTrans) [48]. It encompasses data from 325 sensors located in the Bay Area and spans a duration of 6 months, from January 1st, 2017 to May 31st, 2017 [36]. The traffic information is recorded at 5-minute intervals, resulting in a total of 52,116 time slices.
- **WEATHER** is a publicly available collection of weather data obtained from ten weather stations in Beijing, China [49]. It contains hourly weather measurements recorded from 3:00 AM to 3:00 PM, covering a time period from January 1st, 2015 to May 31st, 2018, totaling 1188 days. All weather stations share the same set of weather monitoring features. In our experiments, two specific weather features, namely *psur* (surface air pressure in degrees celsius) and *q2m* (specific humidity at a height

of two meters above the ground in grams per kilogram), were selected.

Baselines. We select a wealth of baselines that contain traditional machine learning methods, typical GNN-based methods, and self-adaptation methods designed for multi-stream.

Traditional Methods:

- **HA**: Historical Average, an approach that utilizes a weighted average of past data to perform the prediction task;
- **VAR**: Vector Auto-Regression model;
- **SVR**: Support Vector Regression, which uses a linear support vector to achieve the regression aim.
- **FC-LSTM**: Fully Connected LSTM, indicating that the recurrent neural network is linked to the hidden units of the LSTM modules.

Typical GNN-based Methods:

- **DCRNN** [36]: The Diffusion Convolutional Recurrent Neural Network replaces the fully connected layer in the GRU [45] with a diffusion convolutional layer to model spatiotemporal dependency;
- **Graph-WaveNet** [17]: Graph WaveNet utilizes node embedding to learn an adaptable dependency matrix in the graph convolutional layer, which enables the model to capture both temporal and spatial dependencies effectively;
- **AGCRN** [41]: The Adaptive Graph Convolutional Recurrent Network incorporates two adaptive blocks to capture dependencies between streams: node adaptive learning and data-adaptive graph;
- **DGCRN** [37]: Dynamic Graph Convolutional Recurrent Network integrates dynamic graphs generated by dynamic filters and pre-defined graphs into a dynamic convolutional recurrent network to perform the prediction task;
- **ASTGCN** [39]: Attention-based Spatial-Temporal Graph Convolution Networks introduce a spatial-temporal attention mechanism that works in conjunction with graph convolution;
- **STFGNN** [38]: Spatial-Temporal Fusion Graph Neural Networks is a data-driven approach to generate temporal maps during training to compensate for the deep correlations that spatial maps may not reflect.
- **GTS** [42]: Graph for Time Series utilizes Gumbel sampling to generate the discrete graph structure among multiple data streams;
- **S²TAT** [50]: Synchronous Spatiotemporal Graph Transformer simultaneous modelling of data by integrating attention mechanisms and graph convolutions in the proposed S²TAT block.

Self-adaptation Methods Designed for Multi-stream:

- **MuNet** [11]: A concept drift self-adaptation framework achieved by combining a base predictor and Bayesian connectors. The algorithm can only be applied to small-scale datasets, so we choose ten streams for the traffic datasets for testing.
- **SAGN** [35]: A GNN-based self-adaptation framework for multi-stream under concept drift, performed by sub-graph online learning and the periodic validation and reselection of models stored in a GNN pool.

TABLE III
THE PERFORMANCE COMPARISON BETWEEN MSGR AND THE BASELINE METHODS ON REAL-WORLD DATASETS

	Time Step	Metric	HA	VAR	SVR	FC-LSTM	DCRNN	Graph WaveNet	AGCRN	DGCRN	ASTGCN	STFGNN	GTS	S ² TAT	MuNet	SAGN	MSGR
METR-LA	Step 3	MAE	4.16	4.42	3.99	3.44	2.77	2.69	2.87	2.62	4.86	3.23	2.64	2.78	10.02	2.46	2.42
		RMSE	7.80	7.89	8.45	6.30	5.38	5.15	5.58	5.01	9.27	7.43	4.95	5.43	10.02	4.41	4.34
		MAPE	13.00%	10.20%	9.30%	9.60%	7.30%	6.90%	7.70%	6.63%	9.21%	7.82%	6.80%	7.38%	15.73%	6.41%	6.26%
	Step 6	MAE	4.16	5.41	5.05	3.77	3.15	3.07	3.23	2.99	5.43	4.02	3.01	3.10	10.06	2.79	2.72
		RMSE	7.80	9.13	10.87	7.23	6.45	6.22	6.58	6.05	10.61	9.49	5.85	6.39	10.06	5.15	5.04
		MAPE	13.00%	12.70%	12.10%	10.90%	8.80%	8.37%	9.00%	8.02%	10.13%	10.00%	8.20%	8.70%	15.79%	7.64%	7.41%
	Step 12	MAE	4.16	6.52	6.72	4.37	3.60	3.53	3.62	3.44	6.51	5.05	3.41	3.43	10.13	3.16	3.07
		RMSE	7.80	10.11	13.76	8.69	7.59	7.37	7.51	7.19	12.52	11.67	6.74	7.32	10.13	5.91	5.77
		MAPE	13.00%	15.80%	16.70%	13.20%	10.50%	10.10%	10.38%	9.73%	11.64%	12.97%	9.90%	10.02%	15.97%	9.10%	8.79%
PEMS-BAY	Step 3	MAE	2.88	1.74	1.85	2.05	1.38	1.30	1.37	1.28	1.52	1.39	1.32	1.33	5.26	1.30	1.29
		RMSE	5.59	3.16	3.59	4.19	2.95	2.74	2.87	2.69	3.13	2.90	2.62	2.89	5.26	2.43	2.43
		MAPE	6.80%	3.60%	3.80%	4.80%	2.90%	2.73%	2.94%	2.66%	3.22%	2.96%	2.80%	2.83%	18.36%	2.75%	2.71%
	Step 6	MAE	2.88	2.32	2.48	2.20	1.74	1.63	1.69	1.59	2.01	1.77	1.64	1.62	5.31	1.60	1.58
		RMSE	5.59	4.25	5.18	4.55	3.97	3.70	3.85	3.63	4.27	3.97	3.41	3.73	5.31	3.11	3.09
		MAPE	6.80%	5.00%	5.50%	5.20%	3.90%	3.67%	3.87%	3.55%	4.48%	4.02%	3.60%	3.67%	18.46%	3.61%	3.54%
	Step 12	MAE	2.88	2.93	3.28	2.37	2.07	1.95	1.96	1.89	2.61	2.15	1.91	1.85	5.46	1.85	1.81
		RMSE	5.59	5.44	7.08	4.96	4.74	4.52	4.54	4.42	5.42	4.92	3.97	4.30	5.46	3.61	3.55
		MAPE	6.80%	6.50%	8.00%	5.70%	4.90%	4.63%	4.64%	4.43%	6.00%	5.17%	4.40%	4.31%	18.71%	4.37%	4.26%
WEATHER (psur)	Step 3	MAE	28.52	1.56	1.62	2.06	1.06	1.00	1.03	0.94	1.52	1.44	1.04	1.26	3.42	0.96	0.93
		RMSE	39.96	4.60	5.21	5.00	4.81	4.80	4.77	4.33	4.50	5.00	2.86	5.00	3.42	1.29	1.25
		MAPE	3.00%	0.16%	0.16%	0.21%	0.11%	0.10%	0.11%	0.10%	0.14%	0.15%	0.11%	0.13%	0.35%	0.10%	0.09%
	Step 6	MAE	28.52	2.30	2.57	2.86	1.51	1.44	1.48	1.56	2.15	2.03	1.55	1.83	3.45	1.43	1.38
		RMSE	39.96	5.36	6.29	5.80	5.23	5.13	5.11	5.09	5.32	5.33	3.39	4.74	3.45	1.79	1.73
		MAPE	3.00%	0.23%	0.26%	0.29%	0.16%	0.15%	0.15%	0.16%	0.22%	0.21%	0.16%	0.19%	0.35%	0.15%	0.14%
	Step 12	MAE	28.52	3.09	3.58	3.71	2.37	2.25	2.31	2.71	2.90	2.79	2.49	2.59	3.58	2.29	2.21
		RMSE	39.96	6.02	7.21	6.54	5.24	5.57	5.60	6.11	5.94	5.85	4.30	5.66	3.58	2.67	2.56
		MAPE	3.00%	0.31%	0.36%	0.38%	0.24%	0.23%	0.24%	0.28%	0.30%	0.28%	0.25%	0.26%	0.36%	0.23%	0.23%
WEATHER (q2m)	Step 3	MAE	5.15	0.61	0.25	0.30	0.59	0.56	0.59	0.56	0.61	0.64	0.60	0.63	1.33	0.56	0.55
		RMSE	6.29	1.04	0.58	0.58	1.03	1.00	1.03	1.01	1.02	1.06	0.88	1.12	1.33	0.69	0.68
		MAPE	191.12%	12.64%	13.32%	19.28%	11.20%	10.38%	11.20%	10.89%	13.39%	12.22%	11.26%	12.35%	42.04%	10.72%	10.59%
	Step 6	MAE	5.15	0.86	0.38	0.43	0.82	0.79	0.82	0.81	0.86	0.89	0.84	0.84	1.35	0.79	0.77
		RMSE	6.29	1.34	0.74	0.73	1.34	1.30	1.33	1.34	1.34	1.38	1.17	1.34	1.35	0.93	0.91
		MAPE	191.12%	18.84%	21.32%	29.07%	16.45%	15.30%	16.13%	16.61%	17.63%	16.98%	16.47%	16.10%	42.55%	16.02%	15.60%
	Step 12	MAE	5.15	1.17	0.53	0.59	1.16	1.10	1.16	1.17	1.16	1.16	1.16	1.15	1.55	1.10	1.06
		RMSE	6.29	1.72	0.89	0.88	1.77	1.70	1.74	1.79	1.72	1.73	1.53	1.74	1.55	1.25	1.20
		MAPE	191.12%	27.73%	32.70%	42.29%	24.98%	23.33%	25.00%	25.83%	25.11%	24.46%	24.43%	24.88%	46.65%	24.24%	23.20%

*Bold: Our method.

B. Experimental Setup

Platform. The model implementation is carried out using PyTorch 1.7.1. All experiments are conducted on a server equipped with an Nvidia Quadro RTX 8000 GPU with 48 GB memory. The server's CPU is an Intel(R) Xeon(R) Gold 6226R CPU operating at 2.90 GHz.

Parameter settings. In the training stage, the temperature parameter δ of Gumbel-softmax trick is set to 0.9. The training epoch number is set to 200 and the initial learning rate is set to 0.005 for the two traffic datasets and 0.001 for WEATHER. In the online testing stage, the sub-graph updating rate e_{ol} is set to 0.00001 for PEMS-BAY and 0.000005 for the other datasets. The sub-graph updating epoch number is set to 3 for all datasets when no drift occurs, and 5 for METR-LA under drift. The window size of the proposed dynamic KNN graph is set to 48

for METR-LA and PEMS-BAY, and 36 for WEATHER. The regularization penalty coefficient λ is set to 0.01 for the non-drift situation and 1 for the drift situation. The k for the KNN graph is set to 10 for METR-LA and PEMS-BAY, and 5 for WEATHER.

Evaluation Metrics. We use mean absolute error (MAE), mean absolute percentage error (MAPE) and root mean square error (RMSE) to evaluate the performance of MSGR and all the compared methods. The prediction values and ground truth values are transformed into the original values first and then the evaluation computation process is performed.

C. Experiment Results

We evaluate MSGR on three large-scale real-world datasets. Table III shows the overall comparison results. Our method achieves the best performance in multi-step prediction tasks

TABLE IV
 THE ABLATION EXPERIMENTS

	Step	Metric	DGCRN	w/o ol	w/o adp	w/o λ	MSGR
METR-LA	Step 3	MAE	2.62	2.60	2.47	2.44	2.42
		RMSE	5.01	4.42	4.87	4.37	4.34
		MAPE	6.63%	6.86%	6.41%	6.30%	6.26%
	Step 6	MAE	2.99	2.97	2.80	2.74	2.72
		RMSE	6.05	5.78	5.16	5.07	5.04
		MAPE	8.02%	8.36%	7.69%	7.44%	7.41%
	Step 12	MAE	3.44	3.35	3.17	3.09	3.07
		RMSE	7.19	6.65	5.92	5.80	5.77
		MAPE	9.73%	10.07%	9.10%	8.83%	8.79%

*Bold: Our method.

across all datasets. The PEMS-BAY and WEATHER datasets exhibit more regular changes, while the METR-LA dataset experiences more drifts in the testing stage.

Remarkably, MSGR demonstrates a noteworthy average accuracy improvement over the state-of-the-art traditional GNN-based model by **9.14%** on the METR-LA dataset. Additionally, it outperforms the SAGN model, which incorporates concept drift adaptation, with an average improvement of **2.33%** on the METR-LA dataset. MSGR achieves significant improvements, particularly in far-step prediction tasks. Specifically, for the Horizon 12 prediction task, MSGR achieves an improvement of **11.07%** compared to the traditional GNN-based models and a **2.93%** improvement compared to the SAGN adaptation model.

MSGR accurately predicts results even in datasets (PEMS-BAY and WEATHER) with periodic changes, such as recurring drifts caused by seasons. It leverages the learned correlation graph structure from historical data and adapts to new minor changes in incoming data samples.

D. Ablation Study

To verify the effectiveness of various components of our framework, we conducted ablation experiments on the METR-LA dataset.

w/o ol refers to testing in the offline mode, where both the sub-graph updating and dynamic graph-based regularization modules are removed. As shown in Table V, our method still outperforms the existing methods even when operating completely in the offline mode. This highlights the significance of our novel graph convolutional layer in providing a superior base prediction model for the online testing stage.

w/o adp indicates that the model adjustments rely solely on sub-graph updating when removing the dynamic graph-based regularization module. It can be observed that, compared to the offline prediction mode, sub-graph updating is an effective adaptive approach. However, the rough updating method also limits the model's performance. So we design the regularization method to utilize the latest data features to construct the graph structure-based regularization to aid the model in capturing local correlation changes more accurately.

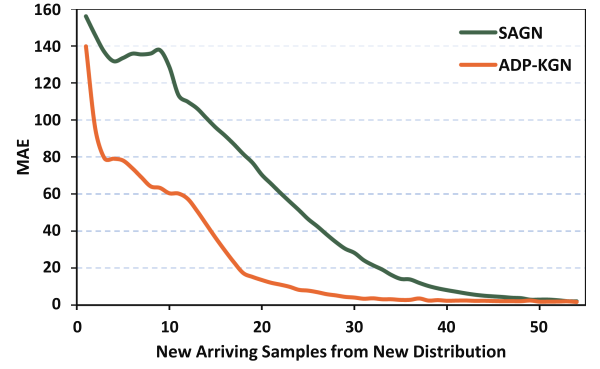


Fig. 4. The Self-adaptation Performance Comparison.

w/o λ represents that the drift detection module is removed, which means the regularization is applied on the sub-graph updating process in the same weight during testing. The result shows that if the self-adaptation process can be targeted, the model performance can be improved further. And our method achieves the best performance in the testing.

To evaluate the self-adaptation performance of MSGR, we further test our method on synthetic datasets, including more pronounced concept drift situations. Our synthetic datasets are the same as those used in [35]. S^a is generated by the Python package *scikit - multiflow* with no correlation between streams. S^b is derived from the PEMS-BAY dataset that streams are related to each other. The subscript *lir*, *tri*, and *pol* represent the different drift functions (as shown in (12)). Then the datasets used for testing are S_{lir}^a , S_{tri}^a , S_{pol}^a , and S_{lir}^b , S_{tri}^b , S_{pol}^b .

$$\begin{cases} S_{lir}^* = 5S^* + 50 \\ S_{tri}^* = \sin(5S^*) + 50 \\ S_{pol}^* = 0.01(S^*)^{2.5} - 0.1(S^*)^2 + 0.5S^* \end{cases} \quad (12)$$

The testing results are shown in Table V. Compared with SAGN, our method achieves a better average prediction performance in most drift cases. Fig. 4 compares the self-adaptation speed (tested on S_{tri}^b dataset). When the multi-stream faces sudden drift, our method achieves faster adaptation and the model converges more efficiently.

E. Parameter Sensitivity

In this section, we test the parameter sensitivity of our method. We test three parameters in our framework: the sub-graph updating rate e , the k for k NN graph and sliding window size τ for two dynamic graphs. The testing results are shown in Fig. 5.

The sub-graph updating rate e (Fig. 5(a)), represents the degree of learning of the model for new samples. Due to the stable global correlation structure, this parameter cannot be too large, or the model will be at risk of overfitting.

The parameter τ is the size of the sliding window, and it determines how many recently obtained new samples are used to generate the k NN graph and the sampled graph. Using the latest samples as features to complete the regularization process helps

TABLE V
THE SELF-ADAPTATION PERFORMANCE COMPARISON

Dataset	Method	Step 3			Step 6			Step 12		
		MAE	RMSE	MAPE	MAE	RMSE	MAPE	MAE	RMSE	MAPE
$\mathcal{S}_{lir}^a$	SAGN	5.40	6.65	2.73%	5.40	6.64	2.73%	5.43	6.69	2.75%
	MSGR	4.96	6.19	2.42%	4.96	6.19	2.43%	4.98	6.22	2.44%
$\mathcal{S}_{tri}^a$	SAGN	0.67	0.76	1.34%	0.67	0.76	1.34%	0.67	0.76	1.35%
	MSGR	0.67	0.76	1.34%	0.67	0.76	1.34%	0.67	0.76	1.34%
$\mathcal{S}_{pol}^a$	SAGN	0.62	0.74	3.47%	0.62	0.73	3.46%	0.62	0.74	3.49%
	MSGR	0.54	0.68	3.06%	0.54	0.68	3.04%	0.54	0.68	3.04%
$\mathcal{S}_{lir}^b$	SAGN	7.67	10.06	7.27%	9.11	12.48	9.02%	11.30	15.43	11.40%
	MSGR	6.36	8.71	6.11%	8.06	11.28	7.99%	10.67	14.85	10.99%
$\mathcal{S}_{tri}^b$	SAGN	0.95	1.10	1.78%	0.98	1.14	1.84%	1.01	1.16	1.86%
	MSGR	0.83	0.98	1.65%	0.83	0.98	1.67%	0.86	1.01	1.72%
$\mathcal{S}_{pol}^b$	SAGN	3.19	4.52	28.34%	3.67	5.37	39.07%	4.36	6.44	54.13%
	MSGR	3.09	4.36	25.98%	3.60	5.25	37.36%	4.32	6.37	49.38%

*Bold: Our method.

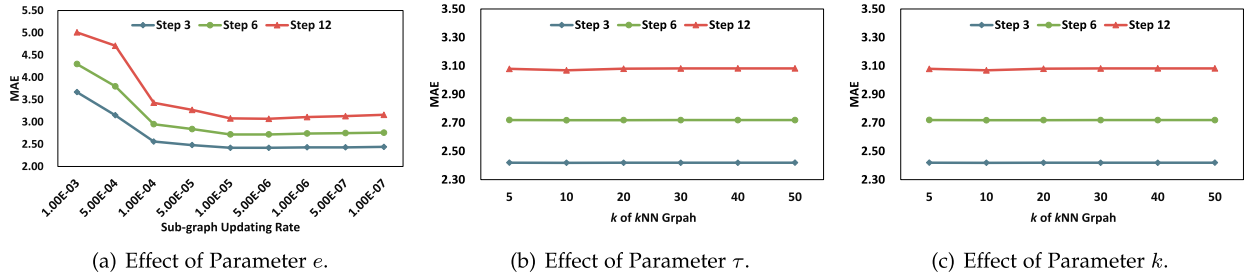


Fig. 5. The effect of different parameters in the self-adaptation stage.

the model learn and adapt correctly to the new data distribution. Through testing (Fig. 5(b)), it can be seen that this parameter is not sensitive. However, considering the update rate of samples, setting this parameter too large may lead to an extension of the model update time.

The k represents the range of nearest neighbours for which we capture instantaneous local correlations with the kNN graph. We set this parameter to be a suitable value according to the specific situation of the practical dataset. It can be observed from Fig. 5(c) that this parameter is not sensitive. Therefore, setting this parameter to a large value is not recommended to avoid excessive noise weighting.

VI. CONCLUSION AND FURTHER STUDY

In this paper, we propose a novel graph structure self-adaptation framework, MSGR, which is designed for multi-stream under concept drift. Our framework first constructs a novel graph neural network architecture to capture deep section-temporal correlations between streams via historical data without pre-defined graphs. A high-performance GNN model is obtained after training and used as the base prediction model to perform the multi-stream multi-step task in online testing. Then, the sub-graph updating and our proposed dynamic graph-based regularization complete the self-adaptation

process. The regularization weight coefficient is dependent on the drift detection results, meaning that a larger regularization weight corresponds to cases of drift. This mechanism adjusts and updates the base model to adapt to concept drift in multi-stream. Any distribution and correlation changes are captured in time and accurate prediction results of all streams will be given simultaneously. Detailed and comprehensive experiments demonstrate the effectiveness and superior performance of MSGR.

In our future study, we will consider more complex drift situations in multi-stream, for example, a multi-stream that contains a lot of noise or less historical data be used for training. Also, the improvement of adaptation speed will be another focus in the future.

REFERENCES

- [1] S. Agrahari and A. K. Singh, "Concept drift detection in data stream mining: A literature review," *J. King Saud Univ.- Comput. Inf. Sci.*, vol. 34, no. 10, pp. 9523–9540, 2022.
- [2] Y. Gao, S. Chandra, Y. Li, L. Khan, and T. Bhavani, "SACCOS: A semi-supervised framework for emerging class detection and concept drift adaption over data streams," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 3, pp. 1416–1426, Mar. 2022.
- [3] M. Jain, G. Kaur, and V. Saxena, "A k-means clustering and SVM based hybrid concept drift detection technique for network anomaly detection," *Expert Syst. Appl.*, vol. 193, 2022, Art. no. 116510.
- [4] R. G. F. Soares and L. L. Minku, "OSNN: An online semisupervised neural network for nonstationary data streams," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 9, pp. 6029–6041, Sep. 2023.

- [5] A. Abbasi, A. R. Javed, C. Chakraborty, J. Nebhen, W. Zehra, and Z. Jalil, "ElStream: An ensemble learning approach for concept drift detection in dynamic social Big Data stream learning," *IEEE Access*, vol. 9, pp. 66408–66419, 2021.
- [6] D. M. V. Sato, S. C. De Freitas, J. P. Barddal, and E. E. Scalabrín, "A survey on concept drift in process mining," *ACM Comput. Surv.*, vol. 54, no. 9, pp. 1–38, 2021.
- [7] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Comput. Surv.*, vol. 46, no. 4, pp. 1–37, 2014.
- [8] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, and G. Zhang, "Learning under concept drift: A review," *IEEE Trans. Knowl. Data Eng.*, vol. 31, no. 12, pp. 2346–2363, Dec. 2019.
- [9] Y. Sun, K. Tang, L. L. Minku, S. Wang, and X. Yao, "Online ensemble learning of data streams with gradually evolved classes," *IEEE Trans. Knowl. Data Eng.*, vol. 28, no. 6, pp. 1532–1545, Jun. 2016.
- [10] P. Castro-Cabrera, G. Castellanos-Dominguez, C. Mera, L. Franco-Marín, and M. Orozco-Alzate, "Adaptive classification using incremental learning for seismic-volcanic signals with concept drift," *J. Volcanol. Geothermal Res.*, vol. 413, 2021, Art. no. 107211.
- [11] M. Zhou, Y. Song, G. Zhang, B. Zhang, and J. Lu, "An efficient Bayesian neural network for multiple data streams," in *Proc. Int. Joint Conf. Neural Netw.*, Shenzhen, China, 2021, pp. 1–8.
- [12] A. Haque, H. Tao, S. Chandra, J. Liu, and L. Khan, "A framework for multistream regression with direct density ratio estimation," in *Proc. AAAI Conf. Artif. Intell.*, 2018.
- [13] E. Yu, Y. Song, G. Zhang, and J. Lu, "Learn-to-adapt: Concept drift adaptation for hybrid multiple streams," *Neurocomputing*, vol. 496, pp. 121–130, 2022.
- [14] B. Dong, Y. Li, Y. Gao, A. Haque, L. Khan, and M. M. Masud, "Multistream regression with asynchronous concept drift detection," in *Proc. IEEE Int. Conf. Big Data*, Boston, MA, USA, 2017, pp. 596–605.
- [15] S. Chandra, A. Haque, L. Khan, and C. Aggarwal, "An adaptive framework for multistream classification," in *Proc. 25th ACM Int. Conf. Inf. Knowl. Manage.*, Indianapolis, IN, USA, 2016, pp. 1181–1190.
- [16] Y.-F. Li, Y. Gao, G. Ayoade, H. Tao, L. Khan, and B. Thuraisingham, "Multistream classification for cyber threat data with heterogeneous feature space," in *Proc. World Wide Web Conf.*, San Francisco, CA, USA, 2019, pp. 2992–2998.
- [17] Z. Wu, S. Pan, G. Long, J. Jiang, and C. Zhang, "Graph wavenet for deep spatial-temporal graph modeling," in *Proc. 28th Int. Joint Conf. Artif. Intell.*, Vienna, Austria, 2019, pp. 1907–1913.
- [18] Z. Shao, Z. Zhang, F. Wang, and Y. Xu, "Pre-training enhanced spatial-temporal graph neural network for multivariate time series forecasting," in *Proc. 28th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, Washington DC, USA, 2022, pp. 1567–1577.
- [19] J. James, "Graph construction for traffic prediction: A data-driven approach," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 9, pp. 15015–15027, Sep. 2022.
- [20] A. Bifet and R. Gavaldá, "Learning from time-changing data with adaptive windowing," in *Proc. SIAM Int. Conf. Data Mining*, Minneapolis, Minnesota, USA, 2007, pp. 443–448.
- [21] J. Gama, P. Medas, G. Castillo, and P. Rodrigues, "Learning with drift detection," in *Proc. Braz. Symp. Artif. Intell.*, 2004, pp. 286–295.
- [22] R. S. M. de Barros, J. I. G. Hidalgo, and D. R. de Lima Cabral, "Wilcoxon rank sum test drift detector," *Neurocomputing*, vol. 275, pp. 1954–1963, 2018.
- [23] Ö. Gözüağık and F. Can, "Concept learning using one-class classifiers for implicit drift detection in evolving data streams," *Artif. Intell. Rev.*, vol. 54, pp. 3725–3747, 2021.
- [24] G. Song, Y. Ye, H. Zhang, X. Xu, R. Y. Lau, and F. Liu, "Dynamic clustering forest: An ensemble framework to efficiently classify textual data stream with concept drift," *Inf. Sci.*, vol. 357, pp. 125–143, 2016.
- [25] S. Xu, L. Feng, S. Liu, and H. Qiao, "Self-adaption neighborhood density clustering method for mixed data stream with concept drift," *Eng. Appl. Artif. Intell.*, vol. 89, 2020, Art. no. 103451.
- [26] S. Khezri, J. Tanha, A. Ahmadi, and A. Sharifi, "STDS: Self-training data streams for mining limited labeled data in non-stationary environment," *Appl. Intell.*, vol. 50, pp. 1448–1467, 2020.
- [27] P. Domingos and G. Hulten, "Mining high-speed data streams," in *Proc. 6th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, Boston, MA, USA, 2000, pp. 71–80.
- [28] J. Gama, R. Rocha, and P. Medas, "Accurate decision trees for mining high-speed data streams," in *Proc. 9th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, Washington, DC, USA, 2003, pp. 523–528.
- [29] K. Wang, J. Lu, A. Liu, Y. Song, L. Xiong, and G. Zhang, "Elastic gradient boosting decision tree with adaptive iterations for concept drift adaptation," *Neurocomputing*, vol. 491, pp. 288–304, 2022.
- [30] A. Cano and B. Krawczyk, "Rose: Robust online self-adjusting ensemble for continual learning on imbalanced drifting data streams," *Mach. Learn.*, vol. 111, no. 7, pp. 2561–2599, 2022.
- [31] S. Disabato and M. Roveri, "Tiny machine learning for concept drift," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 6, pp. 8470–8481, Jun. 2024.
- [32] Y. Song, G. Zhang, H. Lu, and J. Lu, "A noise-tolerant fuzzy c-means based drift adaptation method for data stream regression," in *Proc. IEEE Int. Conf. Fuzzy Syst.*, New Orleans, Louisiana, USA, 2019, pp. 1–6.
- [33] L. Yuan et al., "Recent advances in concept drift adaptation methods for deep learning," in *Proc. 31st Int. Joint Conf. Artif. Intell.*, 2022, pp. 5654–5661.
- [34] Y. Song, G. Zhang, H. Lu, and J. Lu, "A fuzzy drift correlation matrix for multiple data stream regression," in *Proc. IEEE Int. Conf. Fuzzy Syst.*, Glasgow, U.K., 2020, pp. 1–6.
- [35] M. Zhou, J. Lu, Y. Song, and G. Zhang, "Multi-stream concept drift self-adaptation using graph neural network," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 12, pp. 12828–12841, Dec. 2023.
- [36] Y. Li, R. Yu, C. Shahabi, and Y. Liu, "Diffusion convolutional recurrent neural network: Data-driven traffic forecasting," in *Proc. Int. Conf. Learn. Representations*, Vancouver, BC, Canada, 2018.
- [37] F. Li et al., "Dynamic graph convolutional recurrent network for traffic prediction: Benchmark and solution," *ACM Trans. Knowl. Discov. Data*, vol. 17, pp. 1–21, 2021.
- [38] M. Li and Z. Zhu, "Spatial-temporal fusion graph neural networks for traffic flow forecasting," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 4189–4196.
- [39] K. Guo, Y. Hu, Z. Qian, Y. Sun, J. Gao, and B. Yin, "Dynamic graph convolution network for traffic forecasting based on latent network of laplace matrix estimation," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 2, pp. 1009–1018, Feb. 2022.
- [40] C. Zheng, X. Fan, C. Wang, and J. Qi, "Gman: A graph multi-attention network for traffic prediction," in *Proc. AAAI Conf. Artif. Intell.*, New York, USA, 2020, pp. 1234–1241.
- [41] L. Bai, L. Yao, C. Li, X. Wang, and C. Wang, "Adaptive graph convolutional recurrent network for traffic forecasting," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 17804–17815.
- [42] C. Shang, J. Chen, and J. Bi, "Discrete graph structure learning for forecasting multiple time series," in *Proc. Int. Conf. Learn. Representations*, 2021.
- [43] E. Jiang, S. Gu, and B. Poole, "Categorical reparameterization with gumbel-softmax," in *Proc. Int. Conf. Learn. Representations*, Toulon, France, 2017.
- [44] C. J. Maddison, A. Mnih, and Y. W. Teh, "The concrete distribution: A continuous relaxation of discrete random variables," in *Proc. Int. Conf. Learn. Representations*, Toulon, France, 2017.
- [45] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," in *Proc. Adv. Neural Inf. Process. Syst.*, 2014.
- [46] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014, *arXiv:1412.6980*.
- [47] H. V. Jagadish et al., "Big data and its technical challenges," *Commun. ACM*, vol. 57, no. 7, pp. 86–94, 2014.
- [48] S. Chavhan and P. Venkataram, "Prediction based traffic management in a metropolitan area," *J. Traffic Transp. Eng.*, vol. 7, no. 4, pp. 447–466, 2020.
- [49] M. Liu, "AI-challenger-weather-forecast," 2019. [Online]. Available: <https://github.com/Ericonaldo/AI-Challenger-Weather-Forecast>
- [50] T. Wang et al., "Synchronous spatiotemporal graph transformer: A new framework for traffic data prediction," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 12, pp. 10589–10599, Dec. 2023.



Ming Zhou (Graduate Student Member, IEEE) received the MS degree in computer application technology from the School of Computer Science and Information Engineering, Hefei University of Technology, Anhui, China, in 2020. He is currently working toward the PhD degree with the Faculty of Engineering and Information Technology, University of Technology Sydney, Australia. He is a member of the Decision Systems and e-Service Intelligence Lab, Australia Artificial Intelligence Institute, University of Technology Sydney. His research interests include concept drift and non-stationary stream learning.



Pengqian Lu received the BE degree from Hangzhou Dianzi University, Zhejiang, China, in 2017, and the master of philosophy engineering degree from the University of Sydney, in 2022. Currently, he is working toward the PhD degree in computer science from the University of Technology Sydney. His research interests include weakly supervised learning and graph neural networks.



Jie Lu (Fellow, IEEE) received the PhD degree from Curtin University, in 2000. She is an Australian laureate fellow, IFSA fellow, distinguished professor and the director of Australian Artificial Intelligence Institute (AAIL), University of Technology Sydney, Australia. Her main research expertise is in transfer learning, concept drift, fuzzy systems, decision support systems and recommender systems. She has been awarded 12 Australian Research Council (ARC) discovery and linkage grants and led 20 industry projects. She has published more than 500 papers

in IEEE Transactions and other leading journals and conferences. Many of her research results have been successfully transferred to industry. She has delivered 40 keynote speeches at international conferences, supervised 50 PhD students to completion, serves as editor-in-chief for Knowledge-Based Systems and received national and international awards including the IEEE Transactions on Fuzzy Systems Outstanding Paper Awards (2018 and 2022).



Guangquan Zhang received the PhD degree in applied mathematics from Curtin University, Australia, in 2001. He is an Australian Research Council (ARC) QEII fellow, associate professor and the director with the Decision Systems and e-Service Intelligent (DeSI) Research Laboratory, Australian Artificial Intelligence Institute, University of Technology Sydney, Australia. From 1993 to 1997, he was a full professor with the Department of Mathematics, Hebei University, China. His main research interests lie in fuzzy multi-objective, bilevel and group decision

making, fuzzy measures, transfer learning and concept drift adaptation. He has published six authored monographs and more than 500 papers including some 300 articles in leading international journals. He has supervised 40 PhD students to completion and mentored 15 Postdoc fellows. He has won ten very competitive ARC Discovery grants and many other research projects. His research has been widely applied in industries.